

Relazione di Progetto di Sistemi Operativi

Applicazione Elenco Telefonico con Socket TCP

Giacomo Abbruciati - Matricola: 0356240

30 luglio 2026

Indice

1	Introduzione, Specifiche e Scelte progettuali	2
1.1	Specifiche del progetto	2
1.2	Scelte progettuali	2
2	Specifica del Protocollo	2
2.1	Struttura Header di Pacchetto	2
2.2	Opcode di richiesta	3
2.2.1	Opcode LOGIN & REGISTER	3
2.3	Opcode INSERT	4
2.4	Opcode DELETE	4
2.5	Status code di risposta	4
3	Specifica del Server	5
3.1	Architettura Concorrente	5
3.2	Autenticazione e persistenza	5
3.3	Base di dati e sincronizzazione	5
3.4	Elevazione dei permessi	5
4	Specifica del Client	5
5	Esecuzione e Collaudo	5
5.1	Compilazione e Avvio	5
5.2	Esempio di Interazione	6
6	Conclusioni	6

1 Introduzione, Specifiche e Scelte progettuali

1.1 Specifiche del progetto

La specifica del progetto richiede che venga sviluppato un servizio di "Elenco Telefonico", supportato da un server che gestisca le richieste provenienti da client residenti su macchine diverse.

L'applicazione sviluppata dovrà supportare la ricerca e l'aggiunta di record all'interno dell'elenco telefonico se si è sufficientemente autorizzati.

La specifica richiede inoltre lo sviluppo di un'applicazione client in grado di interagire con il server per effettuare le operazioni richieste.

1.2 Scelte progettuali

Per l'interazione tra il client ed il server si è scelto di sviluppare un protocollo la cui specifica è descritta nella sezione successiva.

Il client ed il server scambiano messaggi tramite *Socket TCP*: questo permette alle due applicazioni di risiedere su macchine diverse collocate in aree geografiche diverse.

Poiché nella specifica viene richiesto che gli utenti che sono autorizzati ad inserire contatti non siano generalmente gli stessi autorizzati a ricercarli, si è optato per una struttura che prevede dei livelli di autorizzazione per la gestione delle operazioni effettuabili.

Ruolo	Ricerca contatti	Inserimento contatti	Eliminazione contatti
Anonymous	✗	✗	✗
User	✓	✗	✗
Admin	✓	✓	✓

Tabella 1: Descrizione dei Permessi Utente

Quando un utente si registra, a questo viene assegnato automaticamente il ruolo *Anonymous*: questo non permette all'utente di svolgere alcuna operazione.

Il server tramite la sua console ha la possibilità di modificare i permessi di un utente, cambiando i privilegi.

Come descritto nella Tabella 1 un utente di tipo *User* ha l'unica possibilità di ricercare contatti nell'elenco telefonico mentre per l'inserimento è necessario un livello di autorizzazione *Admin*.

2 Specifica del Protocollo

Il protocollo che si è deciso di sviluppare si appoggia sul protocollo di trasporto *TCP* per la garanzia di consegna dei pacchetti.

Si è optato per uno schema *Header – Payload* con un *Header* fisso a 32 bit e un *Payload* variabile a seconda del tipo di messaggio. L'*Header* contiene un marcatore d'inizio per scartare immediatamente le richieste malformate; questo contiene anche il tipo di messaggio inviato e la lunghezza di un eventuale *Payload*.

2.1 Struttura Header di Pacchetto

L'header di pacchetto contiene le informazioni relative alla natura del messaggio e alla lunghezza del relativo payload.

Il primo byte rappresenta una sequenza di riconoscimento del pacchetto, per verificare che la provenienza della richiesta sia stata originata da un client adeguato, filtrando così messaggi errati causati ad esempio dalla connessione tramite *HTTP* al server. Questo byte è stato fissato a **0xA5**.

Il secondo byte rappresenta l'*Opcode* (0x00–0x04) della richiesta se il messaggio è in ingresso al server, oppure lo *Status Code* (0x80–0x89) se il messaggio è in uscita dal server.

I successivi due byte indicano la lunghezza del *Payload*, ovvero il contenuto effettivo del messaggio.

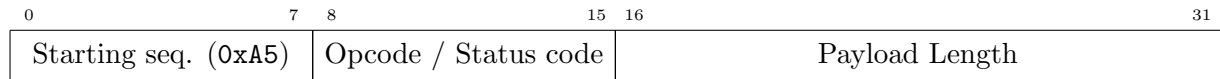


Figura 1: Struttura dell'Header di Pacchetto

2.2 Opcode di richiesta

Una richiesta ha come primo byte informativo l'*Opcode*: questo definisce il tipo di operazione che si intende eseguire, nella Tabella 2 sono mostrate le operazioni disponibili.

Hex	Opcode	Parametri	Descrizione
0x00	LOGIN	<username> <password_hash>	Effettua l'accesso
0x01	REGISTER	<username> <password_hash>	Registra un utente
0x02	INSERT	<name> <phone_number>	Inserisce un contatto
0x03	DELETE	<name>	Rimuove un contatto
0x04	SEARCH	<name>	Ricerca un contatto

Tabella 2: Lista di opcode disponibili

2.2.1 Opcode LOGIN & REGISTER

Gli *Opcode* LOGIN e REGISTER individuano rispettivamente le operazioni di Accesso e Registrazione al servizio di elenco telefonico. Il *Payload* è strutturato ponendo nei primi Payload Length–32 byte l'username dell'utente e negli ultimi 32 byte l'hash della password effettuato dal client.

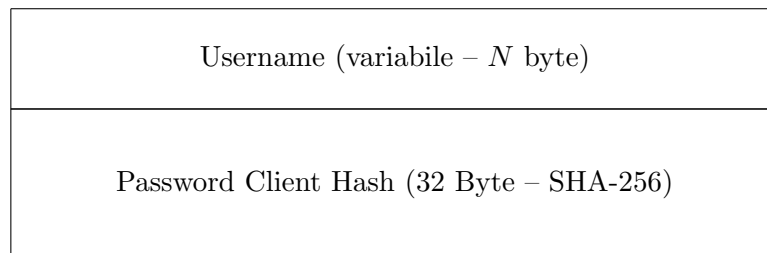


Figura 2: Struttura del payload di una richiesta LOGIN o REGISTER.

A seguito di una richiesta di tipo LOGIN, se l'autenticazione avrà successo, il client aspetterà una risposta per cui il primo e unico byte del *Payload* conterrà il livello di autorizzazione dell'utente che ha effettuato l'accesso. Nel caso l'autenticazione non abbia successo lo *Status Code* sarà INVALID_CREDENTIALS il *Payload* della risposta avrà dimensione nulla.

Per una richiesta di tipo REGISTER ciò non avviene poiché un utente appena registrato si presuppone avere il livello di autorizzazione minimo, per cui in questo caso il *Payload* avrà dimensione nulla.



(a) Struttura pacchetto di risposta al LOGIN con esito positivo.



(b) Struttura pacchetto di risposta alla REGISTER o ad una LOGIN senza successo.

2.3 Opcode INSERT

L'*Opcode* INSERT individua l'operazione di inserimento di un nuovo contatto nell'Elenco telefonico. Il primo byte del *Payload* di questo tipo di pacchetto individua la lunghezza in byte del nome del contatto da inserire, i seguenti N byte saranno quindi costituiti dal nome di questo mentre i restanti byte saranno attribuiti al numero di telefono.

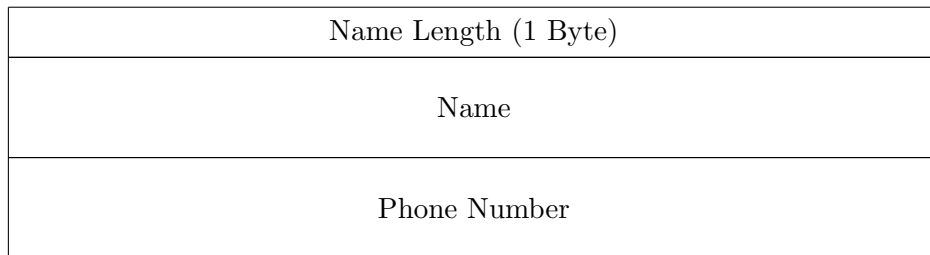


Figura 4: Struttura del payload di una richiesta INSERT.

Una richiesta di tipo INSERT che ha successo sarà seguita da una risposta con *Status code* OK. Se l'utente non dovesse essere autorizzato ad eseguire questa operazione la risposta avrà uno *Status Code* UNAUTHORIZED. In entrambi i casi il *Payload* sarà vuoto.

2.4 Opcode DELETE

L'*Opcode* DELETE individua l'operazione di rimozione di un contatto dall'Elenco telefonico. Il *Payload* di questo tipo di pacchetto contiene unicamente il *Name* del contatto che si intende eliminare: questo deve coincidere esattamente con il *Name* del contatto target.

Una richiesta di tipo DELETE che ha successo sarà seguita da una risposta con *Status code* OK. Se l'utente non dovesse essere autorizzato ad eseguire questa operazione la risposta avrà uno *Status Code* UNAUTHORIZED. Se invece il contatto non dovesse esistere ci si aspetterà una risposta il cui *Status Code* sarà NOT FOUND. In tutti i casi il *Payload* sarà vuoto.

2.5 Status code di risposta

Hex	Status	Descrizione	Payload
0x80	OK	Operazione completata con successo	<i>Auth level</i> (1 byte) / —
0x81	SERVER_ERROR	Errore interno al server	—
0x82	MALFORMED_REQUEST	Richiesta non valida	—
0x83	NOT_FOUND	Risorsa non trovata	—
0x84	UNAUTHORIZED	Utente non autorizzato	—
0x85	INVALID_CREDENTIALS	Credenziali inserite non valide	—
0x86	USERNAME_TAKEN	Username già registrato	—
0x87	ALL_RETURNED	Ricerca completata (risultati totali)	Elenco contatti trovati
0x88	FEWER_RETURNED	Ricerca completata (risultati parziali)	Elenco contatti trovati
0x89	CONTACT_ALREADY_EXISTS	Contatto già presente	—

Tabella 3: Codici di stato restituiti dal Server e relativo Payload.

3 Specifica del Server

3.1 Architettura Concorrente

Il server è stato progettato per gestire concorrentemente più connessioni: il thread principale è in ascolto sulla porta adibita al servizio (5050), una volta stabilita la connessione, viene creato un thread apposito che gestirà la comunicazione con il client.

3.2 Autenticazione e persistenza

L'autenticazione di un utente avviene tramite una richiesta di tipo **LOGIN** al server, l'autenticazione persiste per tutta la connessione fino alla disconnessione del client.

Nel caso in cui il client inoltri un'ulteriore richiesta di **LOGIN** l'autenticazione se di successo cambierà l'identità del client.

3.3 Base di dati e sincronizzazione

Il salvataggio dei dati (credenziali degli utenti e contatti) avviene rispettivamente su due file su disco. Poiché le operazioni su questi possono avvenire concorrentemente, viene utilizzato un read-write lock (`pthread_rwlock_t`): questo permette a più thread di leggere concorrentemente i file, mettendo in attesa eventuali scrittori che potranno effettuare le operazioni desiderate solo al termine di tutte le letture.

Quest'architettura espone però al rischio di starvation da parte di scrittori eventuali che potrebbero non ottenere mai il loro turno, per questo è stata utilizzata l'opzione

`PTHREAD_RWLOCK_PREFER_WRITER_NONRECURSIVE_NP`

Tale attributo fornisce priorità ad una scrittura rispetto ad una lettura, bloccando l'acquisizione del lock da parte di un lettore nel caso in cui una richiesta di scrittura sia pendente.

Va osservato però che una scrittura è statisticamente comunque meno frequente di una lettura, utilizzando questa opzione si accetta quindi un possibile ritardo da parte di un lettore, eliminando però del tutto l'eventualità di starvation per uno scrittore.

3.4 Elevazione dei permessi

4 Specifica del Client

Il client è un'applicazione da riga di comando che:

1. Apre una socket TCP verso l'IP e la porta del server.
2. Mostra un menu o legge i comandi dell'utente da `stdin`.
3. Formatta la richiesta secondo il protocollo e la invia sulla socket.
4. Attende la risposta del server, ne fa il parsing e mostra il risultato all'utente.

5 Esecuzione e Collaudo

5.1 Compilazione e Avvio

```
# Avvio del Server
$ ./server 8080

# Avvio del Client
$ ./client 127.0.0.1 8080
```

5.2 Esempio di Interazione

Un tipico flusso di comunicazione tra Client (C) e Server (S):

```
C: SEARCH Mario
S: 200 OK 3331234567

C: SEARCH Giuseppe
S: 404 NOT_FOUND

C: QUIT
S: 200 BYE
```

6 Conclusioni

Il progetto ha permesso di verificare l'efficacia della gestione delle socket in ambiente di rete e la semplicità di definizione di un protocollo applicativo su misura per servire richieste concorrenti.